

EECE 306 Lab 1 notebook. Building the Toolbox, Constants and Vector Algebra

Team 3: Amelia and Carissa. TODO replace with your team number and member names.

Fill in every item marked TODO, then run `publish('Lab01_notebook.m')` from the toolbox root and print the resulting HTML to PDF. Three to five pages. The notebook is graded on required results, verification evidence, interpretation, honesty about problems, and presentation.

Contents

- [Setup](#)
- [Part I. Physical constants](#)
- [Part II. Vector algebra on the required checks](#)
- [Part II continued. Shape discipline](#)
- [Test Output](#)
- [Interpretation](#)
- [Problems encountered](#)
- [Full test suite](#)

Setup

This cell must run clean from the toolbox root.

```
clear; close all;
disp(version)
```

24.2.0.3212159 (R2024b) Update 9

Part I. Physical constants

The two derived constants must be computed, not typed in.

```
fprintf('eps0 = %.10e F/m\n', em.const.eps0());
fprintf('mu0 = %.10e H/m\n', em.const.mu0());
fprintf('c0 = %.10e m/s\n', em.const.c0());
fprintf('eta0 = %.6f ohm\n', em.const.eta0());
em.test.assertClose(em.const.c0(), 1/sqrt(em.const.mu0()*em.const.eps0()), 1e-14, 'c0 derived');
em.test.assertClose(em.const.eta0(), sqrt(em.const.mu0()/em.const.eps0()), 1e-14, 'eta0 derived');
disp('Derived constant checks passed.')
```

```
eps0 = 8.8541878128e-12 F/m
mu0 = 1.2566370614e-06 H/m
c0 = 2.9979245808e+08 m/s
eta0 = 376.730314 ohm
Derived constant checks passed.
```

Part II. Vector algebra on the required checks

```
fprintf('mag([3 4 0]) = %.15g (expect 5 exactly)\n', em.vec.mag([3 4 0]));
disp('unit([3 4 0]) ='); disp(em.vec.unit([3 4 0]))
disp('unit([0 0 0]) (expect all NaN) ='); disp(em.vec.unit([0 0 0]))
fprintf('angle perpendicular = %.15g (expect pi/2)\n', em.vec.angle([1 0 0],[0 1 0]));
ap = em.vec.angle([1 0 0],[-1 0 0]);
fprintf('angle antiparallel = %.15g, real = %d (expect pi, 1)\n', ap, isreal(ap));
disp('fromTo([1 1 1],[2 3 4]) ='); disp(em.vec.fromTo([1 1 1],[2 3 4]))
```

```
mag([3 4 0]) = 5 (expect 5 exactly)
unit([3 4 0]) =
    0.6000    0.8000         0
```

```
unit([0 0 0]) (expect all NaN) =
    NaN    NaN    NaN
```

```
angle perpendicular = 1.5707963267949 (expect pi/2)
angle antiparallel = 3.14159265358979, real = 1 (expect pi, 1)
fromTo([1 1 1],[2 3 4]) =
     1     2     3
```

Part II continued. Shape discipline

Every function must accept Nx3 input and return the documented shape.

```
A = randn(100,3); B = randn(100,3);
fprintf('mag    input 100x3 output %s\n', mat2str(size(em.vec.mag(A))));
fprintf('unit    input 100x3 output %s\n', mat2str(size(em.vec.unit(A))));
fprintf('angle    input 100x3 output %s\n', mat2str(size(em.vec.angle(A,B))));
fprintf('fromTo  input 100x3 output %s\n', mat2str(size(em.vec.fromTo(A,B))));
```

```
mag    input 100x3 output [100 1]
unit    input 100x3 output [100 3]
angle    input 100x3 output [100 1]
fromTo  input 100x3 output [100 3]
```

Test Output

PASS test_lab01 0.10 s

 1 passed, 0 failed, 1 total
 All tests passing.

Interpretation

TODO three to six sentences in your own words. Why must `c0` and `eta0` be computed from `eps0` and `mu0` rather than typed in as numbers, and why is `==` the wrong way to compare a computed field value against a formula.

1. Since `c0` and `eta0` are constants, it is important to have as much precision as possible, especially when many significant digits are needed. Using a number instead of a function will introduce rounding errors that can significantly alter results. 2. The comparison operator `=="` is not effective for comparing a computed field value against a formula because floating point numbers calculated by the system may not match the formula exactly due to error, and thus using `=="` will evaluate the statement to "False."

Problems encountered

TODO state plainly anything that did not work and what was tried. An honest account of a failure earns more credit than silence. Write NONE if the session was clean.

I tried computing the interior angle by writing the equation as one line: `th = arccos((A .* B) ./ (em.vec.mag(A) .* em.vec.mag(B)))`. This did produce a result, but the output (when testing) was a 100x3 vector instead of a 100x1 vector. I had to compute each value separately, write the cosine, clamp it, and then use `arccos` on that value to determine theta. I also tried running tests inside the test function, which gave me an infinite loop... it took me a bit before I realized I needed to type in the prompt in the command window. Also, this lab I learned that you need `.*`, `./`, and `.^` to multiply, divide, and raise to a power for element-by-element operations instead of `*`, `/`, and `^`!

Full test suite

```
runTests
```

```
EECE 306 toolbox test suite
-----
24.2.0.3212159 (R2024b) Update 9
eps0 = 8.8541878128e-12 F/m
mu0  = 1.2566370614e-06 H/m
c0   = 2.9979245808e+08 m/s
eta0 = 376.730314 ohm
Derived constant checks passed.
mag([3 4 0]) = 5 (expect 5 exactly)
unit([3 4 0]) =
    0.6000    0.8000         0

unit([0 0 0]) (expect all NaN) =
    NaN    NaN    NaN

angle perpendicular = 1.5707963267949 (expect pi/2)
angle antiparallel = 3.14159265358979, real = 1 (expect pi, 1)
fromTo([1 1 1],[2 3 4]) =
    1    2    3

mag    input 100x3 output [100 1]
unit   input 100x3 output [100 3]
angle  input 100x3 output [100 1]
fromTo input 100x3 output [100 3]
PASS   test_lab01          0.01 s
-----
1 passed, 0 failed, 1 total
All tests passing.
```